Music Replayability Analysis

Predicting Song Replay Behavior from Metadata, Audio Features, and Lyrics

CIS 2450: Big Data Analytics | Spring 2026 Final Project

Chinmay Govind (cgovind@seas.upenn.edu) • Rohith Yelisetty (rohithy@seas.upenn.edu)

April 30, 2026

1. Project Summary

This project investigates the factors that make songs highly replayable, using data collected from three public music APIs and a large-scale lyrics dataset. The target variable, **repeat_listens**, is derived as the difference between total listen count and total unique listener count from ListenBrainz, representing listens beyond a song's first play.

The pipeline spans the full data science lifecycle: API-driven data collection, multi-source joining and feature engineering, exploratory data analysis, regression and classification modeling, NLP-based lyrics analysis, and an interactive Streamlit dashboard. The best regression model (tuned Gradient Boosting) achieved a holdout $R^2 = 0.3118$, showing that structured metadata alone carries meaningful signal for predicting replayability.

Key Findings:

- **55,579** unique tracks after cleaning, across **20 genres** and **14,555** artists
- **3 API sources:** MusicBrainz (metadata) + ListenBrainz (popularity) + AcousticBrainz (audio)
- **1 offline source:** Kaggle 5M Song Lyrics Dataset (lyrics text analysis)
- **20,759** songs matched to lyrics for NLP analysis
- Best model: Gradient Boosting (tuned) — $R^2 = 0.3118$, RMSE = 2.37

2. Codebase Overview

The codebase is organized as a set of standalone Python modules, each responsible for one stage of the pipeline. All shared paths and constants live in `config.py` so no path is hardcoded elsewhere.

2.1. File Tree

```
CIS2450FinalProject/
config.py - shared constants, paths, feature lists, targets
data_collection.py - API fetching:  MusicBrainz, ListenBrainz, AcousticBrainz
data_processing.py - join, feature engineering, DuckDB SQL analytics
eda.py - 8 EDA plots + Kruskal-Wallis and Spearman tests
models.py - 7 regression models, PCA, hyperparameter tuning, imbalance demo
```

```
lyrics_analysis.py - lyrics loading, NLP features, LDA, 10 plots
dashboard.py - Streamlit dashboard - 6 tabs, live prediction console
requirements.txt - pinned Python dependencies

cache/ - pickle caches for API responses and lyrics
musicbrainz_cache.pkl
listenbrainz_cache.pkl
acousticbrainz_cache.pkl
kaggle_lyrics_cache.pkl - built on first run of lyrics_analysis.py

outputs/ - all generated artifacts
music_dataset_processed.csv - final 55 k-row processed dataset
eda1_*.png ... eda8_*.png - EDA visualizations
lyrics1_*.png ... lyrics10_*.png - lyrics NLP visualizations
model_results.csv - CV + holdout metrics for all 7 models
best_gbm_model.joblib - serialized production model
lyrics_features.csv - per-song NLP features
eda_summary.md   model_summary.md   lyrics_summary.md

lyrics/ - place the Kaggle CSV here before running lyrics_analysis.py
ds2.csv - 5M Song Lyrics Dataset (9.2 GB, not committed to git)

report/
report.tex - this document
```

2.2. Module Descriptions

File	Responsibility
`config.py`	Single source of truth for all path constants (`PROCESSED_CSV`, `CACHE_DIR`, etc.), feature lists (`NUMERIC_FEATURES`, `AUDIO_NUMERIC_FEATURES`), and modeling targets (`TARGET = "log_repeat_listens"`).
`data_collection.py`	Fetches recordings from MusicBrainz by genre tag (up to 5 000 per genre), enriches them with ListenBrainz popularity metrics in batches of 500, and optionally adds AcousticBrainz audio features via a multi-threaded fetcher. All results are pickled for fast re-runs.
`data_processing.py`	Deduplicates MusicBrainz rows, performs a 3-way join (MusicBrainz + ListenBrainz + AcousticBrainz) in Polars, engineers new features, winsorizes outliers, label-encodes categoricals, and runs 8 analytical SQL queries in DuckDB. Writes the final processed CSV and a JSON data summary.
`eda.py`	Generates 8 diagnostic plots (target distribution, genre ranking, decade trend, tempo/danceability, correlation heatmap, duration buckets, missingness, outliers by genre) and runs Kruskal-Wallis and Spearman statistical tests.

`models.py`	Trains 7 models (Linear Regression baseline, Ridge, Lasso, PCA+Ridge, Random Forest, Gradient Boosting, and a tuned GBM via RandomizedSearchCV), evaluates with 5-fold CV and holdout metrics, extracts feature importances, runs a classification imbalance demo, and serializes the best model.
`lyrics_analysis.py`	Streams the 9.2 GB Kaggle CSV in 50 k-row chunks to match all processed songs by normalized artist/title, caches results as a pickle, then computes VADER sentiment, type-token ratio, repetitiveness, Flesch readability, bigram frequencies, word length statistics, and rare-word ratios (via `wordfreq`). Fits an 8-topic LDA model and produces 10 plots.
`dashboard.py`	Streamlit application with 6 tabs: Command Center (dataset overview), Genre Lab (interactive drill-down), Model Studio (model comparison and feature importance), Prediction Console (live scoring form), Lyrics Analysis (all NLP plots and tables), and Song Explorer (per-song autocomplete lookup with live Genius API lyric fetch and per-song NLP breakdown).

3. Dataset Overview

3.1. Source 1 — MusicBrainz (Recording Metadata)

MusicBrainz is a community-maintained music encyclopedia with a free, open REST API (`musicbrainz.org/ws/2/r` For each of 20 genre tags the project issues paginated search queries, collecting up to 5 000 recordings per genre. Each recording provides: MBID (a permanent unique identifier), title, duration, release year, release type (Album / Single / EP), and artist metadata (name, type, country, career start year).

Songs that appear under multiple genre searches are deduplicated on MBID; all their genre tags are retained and collapsed into a pipe-separated string.

3.2. Source 2 — ListenBrainz (Popularity Signals)

ListenBrainz tracks listening activity for millions of users. The popularity endpoint (`api.listenbrainz.org/1/popu` accepts batches of 500 MBIDs and returns `total_listen_count` and `total_user_count` for each. The modeling target is defined as:

$$\texttt{repeat_listens} = \texttt{total_listen_count} - \texttt{total_user_count}$$

representing listens beyond one per unique listener — i.e. genuine replay behavior. Songs with no ListenBrainz record are excluded.

3.3. Source 3 — AcousticBrainz (Audio Features) — Optional Enrichment

AcousticBrainz stored Essentia-derived audio descriptors. The project fetches low-level features per MBID (with 6 concurrent threads) extracting: tempo (BPM), danceability, musical key and scale, average loudness, and dynamic complexity. Coverage is 42.81% of the processed dataset; the

modeling pipeline therefore treats audio features as optional enrichment rather than mandatory inputs.

3.4. Source 4 — Kaggle 5M Song Lyrics Dataset

The Kaggle dataset (*nikhilnayak123/5-million-song-lyrics-dataset*) is a 9.2 GB CSV containing lyrics for approximately 5 million songs. The lyrics analysis module streams it in 50 k-row chunks and matches each row to the processed dataset by normalizing both artist name and title (lowercase, strip punctuation). Matched lyrics are cached as a pickle so the 9.2 GB file is scanned only once. **20,759 songs** were matched.

3.5. Final Dataset Statistics

Attribute	Value
Total tracks (after cleaning)	55,579
Total columns	39
Unique artists	14,555
Genres	20
Median repeat listens	344
Median log repeat listens	5.84
AcousticBrainz coverage	42.81%
Songs matched to lyrics	20,759

4. Data Processing Overview

4.1. Joining and Deduplication

The three API sources are joined in Polars on MBID. MusicBrainz rows are first deduplicated: songs fetched under multiple genres have their genre tags aggregated, and the primary genre is taken as the alphabetically first tag. The `genre_match_count` column records how many genre searches each song appeared in (used as a feature).

4.2. Feature Engineering

The following features are derived during processing:

- **duration_sec** — converted from milliseconds, clipped to [0, 900]
- **release_decade** — floored release year (e.g. 1990, 2000)
- **track_age** — `current_year` − `release_year`
- **artist_career_age** — release year − artist debut year
- **log_repeat_listens** — ln(`repeat_listens` + 1), the regression target
- **has_audio_features** — binary flag for AcousticBrainz coverage
- **audio_feature_missing_count** — count of null audio fields (0 or 4)
- **tempo_x_dance** — interaction term: tempo × danceability
- **career_x_duration_min** — interaction term: career age × duration in minutes
- **is_high_replay** — classification target (top-25% repeat listens = 1)

4.3. Outlier Handling

Continuous features (tempo, danceability, loudness, dynamic complexity, duration_sec) are winsorized at the 1st and 99th percentiles to limit the influence of extreme values without discarding records.

4.4. Categorical Encoding

Six categorical columns (genre, release_type, artist_type, artist_country, key, key_scale) are label-encoded into integer columns suffixed `_enc`. Original string columns are retained for display and filtering in the dashboard.

4.5. Null Handling

Missing values in numeric features are handled inside scikit-learn pipelines using `SimpleImputer` (median strategy), ensuring no statistics are computed on test data and no leakage occurs.

4.6. SQL Analytics with DuckDB

After building the Polars frame, `data_processing.py` registers it as an in-memory DuckDB table and runs 8 SQL queries covering: genre-level summary statistics, decade trends, top artists, country breakdown, duration buckets, genre overlap, and AcousticBrainz coverage. Results are exported as individual CSVs and consumed by the dashboard.

4.7. Entity Linking

The lyrics module performs a form of entity linking by normalizing artist and title strings on both sides (lowercase, strip punctuation, collapse whitespace) and performing an exact lookup against the Kaggle dataset. This links the internal MBID-keyed dataset to an external source that has no MBID column — demonstrating cross-dataset record reconciliation.

5. Models Overview

5.1. Experimental Setup

- **Train/test split:** 80 / 20 stratified on the classification target
- **Cross-validation:** 5-fold on the training split only
- **Imputation:** median imputation inside each fold's pipeline
- **Metrics:** RMSE, MAE, and R^2 for regression; accuracy, F1, ROC-AUC for classification
- **Feature set:** metadata-first; AcousticBrainz features included where available

5.2. Models Trained

1. **Linear Regression** (baseline) — ordinary least squares, no regularization
2. **Ridge Regression** — L2 regularization, $\alpha = 1.0$
3. **Lasso Regression** — L1 regularization, $\alpha = 0.1$
4. **PCA + Ridge** — 16 principal components (95.2% explained variance) then Ridge; explores dimensionality reduction
5. **Random Forest** — ensemble of 200 decision trees

6. **Gradient Boosting** — default scikit-learn GBM
7. **Gradient Boosting (tuned)** — `RandomizedSearchCV` over 5 parameters: learning rate, number of estimators, max depth, subsampling rate, min samples per leaf

5.3. Difficulty Concepts Applied

- **Feature importance** — extracted from tree models (Random Forest and both GBM variants); top signals include genre (classical, indie), duration, track age, and release year
- **PCA for feature selection** — 16 components retained from the full feature matrix, explaining 95.2% of variance; compared against direct regression
- **Hyperparameter tuning** — `RandomizedSearchCV` on the GBM with 20 candidates and 5-fold CV; best params: n_estimators = 500, learning_rate = 0.05, max_depth = 5, subsample = 0.7
- **Ensemble models** — Random Forest and Gradient Boosting both outperform all linear baselines
- **Imbalance handling** — logistic regression trained with and without `class_weight='balanced'`; balanced weights improve F1 from 0.327 to 0.506 at the cost of accuracy (0.766 → 0.691) while ROC-AUC is unchanged (0.737)
- **Feature engineering** — interaction terms (`tempo_x_dance`, `career_x_duration_min`) and derived temporal features (`track_age`, `artist_career_age`, `release_decade`)

6. Results Overview

6.1. Regression Model Comparison

Model	CV R^2	Holdout R^2	Holdout RMSE
Linear Regression (baseline)	0.247	0.232	2.505
Ridge	0.247	0.232	2.505
Lasso	0.247	0.232	2.506
PCA + Ridge	0.185	0.168	2.607
Random Forest	0.311	0.301	2.391
Gradient Boosting	0.307	0.294	2.403
Gradient Boosting (tuned)	**0.321**	**0.312**	**2.372**

The tuned Gradient Boosting model is the clear winner, outperforming all linear baselines by approximately 8 R^2 points. Ensemble methods capture the non-linear interaction between genre, track age, duration, and replay behavior that linear models cannot represent. PCA compression slightly hurts performance here, suggesting the original feature space does not contain severe multicollinearity that needs compressing.

6.2. Top Feature Importances (Tuned GBM)

Feature	Importance
genre: classical	0.197
genre: indie	0.133
duration_sec	0.131
track_age	0.095
release_year	0.089
genre: r&b	0.062
audio_feature_missing_count	0.049
genre: punk	0.044
has_audio_features	0.039
genre: metal	0.023

Genre is the dominant predictor, particularly classical and indie — genres with concentrated, committed listener bases. Duration and track age together suggest that well-aged mid-length songs accumulate the most replays over time.

6.3. EDA Findings

- **Target distribution:** `repeat_listens` is heavily right-skewed; the log transform stabilizes variance and is essential for regression.
- **Genre effect:** Kruskal-Wallis test: $H = 6395$, $p \approx 0$ — replay distributions differ significantly across genres.
- **Duration effect:** Spearman $\rho = 0.042$, $p < 10^{-21}$ — mid-length tracks (3–5 min) replay best; very short and very long tracks lag.
- **Decade trend:** Replayability has shifted over release eras, motivating `release_decade` and `track_age` features.
- **Audio features:** Missingness is concentrated in AcousticBrainz fields (57%), confirming the metadata-first modeling strategy.

6.4. Lyrics Analysis Findings

20,759 songs were matched to lyrics from the Kaggle dataset. Key aggregate findings:

- Average VADER compound sentiment: +0.181 (mildly positive overall)
- Average vocabulary richness (TTR): 0.542
- Average repetitiveness (duplicate-line fraction): 0.231
- Sentiment has the strongest lyric-level correlation with replay: $r = -0.089$ (slightly more negative songs replay more)
- **Top words in high-replay songs:** dead, watch, blood, black, dark, fall, wait
- **Top bigrams in high-replay songs:** "saturday night", "close eyes", "cause dont", "hold hold"
- **LDA topics:** 8 coherent themes emerged — conversational ("don't know"), romantic ("love baby"), rock/soul ("rock girls mama"), spiritual ("god lord blood"), multilingual ("que de ich"), hip-hop ("ain't shit fuck"), and nocturnal imagery ("away night eyes")

7. Dashboard Overview

The dashboard is built with Streamlit (`streamlit run dashboard.py`) and organized into six tabs, each covering a distinct analytical workspace.

Tab	Description
Command Center	Top-line dataset health: track count, artist count, audio coverage, best model card; genre distribution bar chart; decade replayability trend; top-25 artist leaderboard; AcousticBrainz coverage by genre; collapsible EDA and modeling narrative summaries.
Genre Lab	Interactive controls (genre multiselect, decade range slider, scatter sample size) that filter the processed dataset in real time. Shows replayability trend lines per genre, a box-plot spread comparison, a duration scatter, and a filtered artist leaderboard.
Model Studio	Cross-validation vs. holdout R^2 grouped bar chart; holdout RMSE/MAE comparison; interactive feature importance view (selectable model); tuned GBM parameters JSON; PCA summary; actual vs. predicted scatter; imbalance handling results table.
Prediction Console	Live scoring form that accepts a full track profile (genre, release type, artist type, country, key, duration, year, career age, optional audio features) and returns the predicted log repeat listens, estimated raw replays, and dataset percentile.
Lyrics Analysis	Four stat cards (tracks analyzed, avg sentiment, vocabulary richness, repetitiveness); all 10 NLP plots displayed in a 2-column grid; top-30 word frequency comparison table; collapsible full summary report.
Song Explorer	Typeahead selectbox over up to 10 000 songs (top by replay score). Selecting a track shows its metadata cards, checks the local lyrics cache, falls back to the Genius API if not cached, then renders a per-song NLP analysis: sentiment breakdown chart, top-15 words bar chart, vocabulary/repetitiveness/readability metrics, and a full lyrics expander.

7.1. Course Topics Demonstrated in the Dashboard

- **Polars** — all data loading and filtering in the dashboard uses Polars DataFrames
- **SQL (DuckDB)** — 8 analytical queries power the Command Center charts
- **Text representations & NLP** — VADER sentiment, type-token ratio, LDA topic model, bigram frequencies, `wordfreq` rarity scoring
- **Record linking** — MusicBrainz MBID links metadata to popularity; normalized artist/title matching links to the Kaggle lyrics corpus
- **Unsupervised learning** — 8-topic LDA surfaced in the Lyrics Analysis tab
- **Supervised learning** — 7 regression models and a classification demo in Model Studio
- **Joins** — 3-way equi-join on MBID in the processing pipeline
- **Feature engineering** — interaction terms, temporal features, audio enrichment flags

8. Conclusion

This project demonstrates that song replayability can be meaningfully predicted from freely available metadata and listening signals, without requiring proprietary audio analysis. The tuned Gradient Boosting model achieves $R^2 = 0.312$ on the holdout set — a substantial improvement over the linear baseline ($R^2 = 0.232$) — primarily driven by genre, duration, and track age.

Key takeaways:

- Genre is the single strongest predictor of replay behavior, reflecting audience loyalty patterns rather than intrinsic audio properties.
- Duration has a non-linear effect: 3–5 minute tracks replay best, consistent with the industry-standard "radio edit" convention.
- Sentiment has a small but consistent negative correlation with replay ($r = -0.089$): darker lyrical themes are modestly associated with higher replays.
- Vocabulary richness (TTR) and repetitiveness show weaker correlations, suggesting lyrical complexity plays a secondary role to thematic content.
- The LDA topic model reveals that high-replay songs cluster around nocturnal, relationship, and intensity-driven themes, while spiritual and non-English topics dominate lower-replay clusters.

Limitations and future work:

- AcousticBrainz is a defunct service (shut down 2022); 57% of tracks have no audio features, capping the model's ability to exploit tempo and danceability.
- Artist/title normalization achieves ∼37% lyrics match rate — fuzzy matching could recover additional songs.
- The regression target is a proxy: ListenBrainz captures only users who actively scrobble, biasing toward niche and enthusiast audiences.
- Future extensions could incorporate embedding-based lyric representations (e.g. sentence transformers) or time-series replay trajectories to capture temporal popularity decay.

Full codebase at `https://github.com/rohithyelisetty/CIS2450FinalProject` | Run order: `data_collection.py` → `data_processing.py` → `eda.py` → `models.py` → `lyrics_analysis.py` → `streamlit run dashboard.py`